🎸 Empório da Música — Customer Support Agent

A Python prototype of a text-message customer support agent for **Empório da Música**, a fictional musical instrument store in Campo Grande, MS (Brazil). Technical challenge — AI Engineer, Artefact.

The agent takes on the store's persona (informal, welcoming, "a friend who knows music"). It answers questions about the **catalog, prices, promotions and stock**, **order status**, **policies** (exchanges, returns, warranty, shipping, payment...) and **store information**, and politely declines anything out of scope.

> **Language note:** the agent talks to customers in **Brazilian Portuguese** (the store's audience and the language of the data and policy manual). This document is in English; the Portuguese original is in `README.md`.

Contents

1. How to run
2. Architecture
3. Technical decisions and rationale
4. Data vs. policies: how the agent decides
5. Assumptions
6. Data issues found
7. Tests
8. Example interactions
9. Known limitations
10. What I would do with more time
11. Use of code assistants

How to run

Requirements: **Python 3.10+** and an API key for any provider that exposes an OpenAI-compatible API (Gemini, OpenAI, Groq...) — or a local model through Ollama (no key needed).

```
git clone <REPOSITORY-URL> && cd emporio-agent
python -m venv .venv && source .venv/bin/activate     # Windows: .venv\Scripts\activate
pip install -r requirements.txt

cp .env.example .env        # edit it and fill in LLM_API_KEY
python main.py              # opens the chat in the terminal
```

In-chat commands: `/debug` (shows every tool call and its result), `/novo` (reset the conversation), `/sair` (quit). Options: `python main.py --session my-chat` resumes/continues a saved conversation; `--debug` starts with tool tracing on.

Model provider (`.env`)

Variable	Default	Description
`LLM_API_KEY`	—	provider key
`LLM_BASE_URL`	`https://generativelanguage.googleapis.com/v1beta/openai/`	OpenAI-compatible endpoint
`LLM_MODEL`	`gemini-2.5-flash`	model name (**check the current name with your provider**)
`AGENT_NOW`	real clock	pins "now" (e.g. `2026-03-25T10:30`) for reproducible demos

`.env.example` includes ready-made recipes for OpenAI, Groq and Ollama (local).

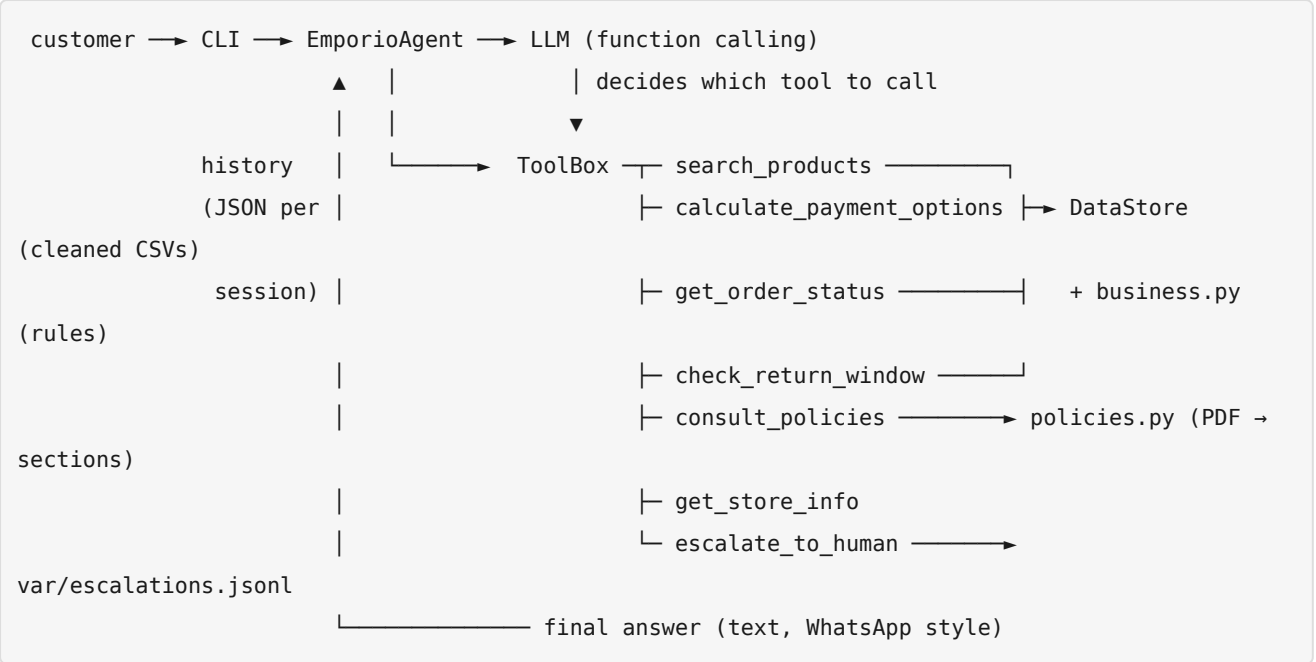
Other commands

```
python -m pytest -q                     # 49 tests, no network and no key needed
python scripts/record_examples.py       # re-records examples/*.md with your LLM
python scripts/record_examples.py 04    # only scenario 04
```

Layout

```
data/                    CSVs + policy manual (PDF)
src/emporio_agent/
  config.py              configuration, provider, store clock
  text.py                text normalization, light stemming, R$ formatting
  data_store.py          CSV loading + cleaning; product search; customer verification
  business.py            manual rules as code: installments, PIX, deadlines, opening hours
  policies.py            PDF → sections parsing + retrieval (topic / BM25)
  tools.py               7 tools (schemas + execution)
  prompts.py             persona, rules, scope
  agent.py               function-calling loop
  memory.py              persisted conversation history
  cli.py                 terminal interface
tests/                   pytest (data, tools, policies, agent loop)
examples/                5 sample conversations
scripts/record_examples.py
```

Architecture



Each turn: message → LLM → (tools → JSON result → LLM)* → answer. The loop is capped at 6 steps and handles tool and provider errors without breaking the conversation.

Technical decisions and rationale

1. Approach: native function calling with business-level tools (hybrid)

Choice: a hand-written *function calling* loop (~60 lines) with 7 specific tools; structured data handled by deterministic code and policies handled by section retrieval.

Why: - The problem has two very different kinds of knowledge: **data that changes** (price, stock, promotion, order) and **rule text** (the manual). Treating both the same way ("throw everything into RAG" or "let the LLM write SQL") gets wrong what matters most — prices and deadlines must be *right*. - **Business tools instead of a SQL agent:** the LLM does not write queries; it passes typed parameters (`category`, `max_price`...). That removes invalid/dangerous SQL and bakes in rules the model would get wrong or forget: a promotion only applies if `is_active`, the 5% PIX discount does not stack on promotional prices, minimum installment value, sold-out / discontinued / coming-soon products, and identity verification (LGPD). - **Math and dates in code, not in the LLM:** installments, discounts, and return deadlines (7/30/90 days, each counted from a different milestone) are computed in `business.py` and tested. - **No LangChain/LlamaIndex:** with 7 tools and no state graph, a framework would only add abstraction and dependencies. An explicit loop keeps limits, logs and errors visible and is easy to explain and debug. (If the flow grew — multi-agent, human approvals, queues — I would reconsider LangGraph.) - **Alternatives considered:** *text ReAct* (less robust than native tool calling and costlier in tokens); *vector RAG over everything* (great for long text, bad for price/stock); *one giant prompt with everything* (the 65 products would fit, but it would go stale, cost more, and have no access control over orders).

2. Model and provider

Choice: an OpenAI-compatible client with `base_url` / `model` set through environment variables. Default: **Gemini 2.5 Flash** through its OpenAI-compatibility endpoint.

Why: the requirement is *function calling* + Portuguese + low latency/cost — a "flash/mini" model is enough, because the "hard" intelligence (rules and math) lives in code. The OpenAI format is the common denominator (OpenAI, Gemini, Groq, OpenRouter, Ollama, vLLM), so **switching provider means editing** `.env`, not code. Gemini has a free tier, which lowers the barrier for whoever runs the project; Ollama covers the fully local scenario. *Honest note:* the code was tested with a fake LLM (automated tests) — end-to-end quality depends on the chosen model; small local models tend to pick tools less reliably.

3. Policy retrieval (the project's "RAG")

Choice: the PDF is read with `pypdf` (*layout* mode, which preserves tables) and split **by the manual's own sections/subsections** (25 chunks). The `consult_policies` tool takes an enumerated `topic` (payment, returns/exchanges, shipping, warranty, promotions, hours, privacy, about the store) and returns the **whole** sections; if no topic fits (`other`), it falls back to **BM25** (implemented in ~20 lines, with light stemming and a small synonym list).

Why: the manual is ~3,000 words. Embeddings + a vector DB would add dependency and cost for no gain, and *get rules with exceptions wrong* (a fixed-size chunk would separate "7 days..." from "...except mouthpieces"). Letting the **LLM pick the topic** (a function with an enum) is more reliable than lexical similarity for short questions; BM25 is the safety net. If the manual grew to dozens of pages, I would swap the fallback for embeddings (keeping section-based chunking).

Internal sections stay out of the index: §7 (tone/flow/special situations) and §10 are instructions for staff, not content to quote to customers. They became **prompt rules** (`prompts.py`).

4. Prompt strategy

- **Persona** anchored in manual §7.1 ("informal but professional, like a friend who knows music"), WhatsApp format (short, no tables/headings, ≤1 emoji, "R$ 1.299,00").
- **Golden rule:** never state price/stock/promotion/deadline/order/hours from memory — always through a tool; if data is missing, say so and offer the team (§7.1: "incorrect information may constitute false advertising").
- An explicit **"question → tool" routing table** (this is how the agent "knows when to query data and when to query policies").
- **§7.3 special situations** encoded as rules: out of stock → alternatives; discontinued → equivalents; expired promotion → transparency, never promise a discount; complaint → empathy + ticket (24 business-hour follow-up).
- **Scope:** instruments only; accessories are declined politely **without inventing partner stores** (the manual asks to suggest partner stores "when possible", but no list exists); off-topic → short refusal and back to business.
- **Robustness:** customer text is treated as *data* (resistance to "ignore the rules / give me 50% / show someone else's orders"); the current date/time is injected every turn.
- **Temperature 0.2** for answers consistent with the data.

5. Interface

CLI (`python main.py`): the challenge focuses on the agent. The logic is decoupled (`EmporioAgent.respond(text) -> AgentReply`), so plugging in a WhatsApp webhook or HTTP API means replacing only `cli.py`. The `/debug` flag shows the tools used — useful for judging the reasoning.

6. History persistence

One JSON file per session in `var/sessions/`, window of the last 20 messages. It persists **only the dialogue** (user/assistant); **tool results are deliberately not saved**: stock, promotions and order status change, so the agent re-queries instead of trusting stale data. Trade-off: "what about the second one?" relies on the assistant having named the product in its previous answer (it does).

7. Data handling (`data_store.py`)

- Loading with stdlib `csv` + dataclasses (the data is tiny; pandas/SQL would be dead weight), whitespace normalization, `specs` (JSON-in-a-string) parsed.
- **Product search** is accent/plural-insensitive (5-letter prefix stemmer: "violão"="violões"; keeps "Kala"≠"Kalani"), joins "gd 20"→"GD20", and has a fuzzy fallback (`difflib`). With a `query`, sold-out/discontinued items **appear flagged** (we never "pretend" they don't exist); when browsing by category/price, unavailable items are hidden and the count is reported.
- **Effective price** = list price − highest **active** promotion (`is_active=1`; not cumulative, §6.2). Ended promotions only feed the "that promotion is over" notice.
- **Inconsistent descriptions are suppressed** (see data issues); name, price and specs are the source of truth.
- **Privacy (LGPD, §9):** orders are opened only with the **e-mail or phone on file**; a failure returns a generic message (it does not reveal whether the order/contact exists); the agent never sees PII beyond the first name. An e-mail shared by two accounts is accepted only together with an order number belonging to one of them.

Data vs. policies: how the agent decides

Customer question	Tool	Source
"How much is the Takamine GD20?" · "guitars under R$1000" · "any promotion?"	`search_products`	CSV (products/promotions/stock)
"How many installments?" · "how much with PIX?"	`calculate_payment_options`	§3 rules in code
"Where is my order?"	`get_order_status` (requires e-mail/phone)	CSV (orders/customers)
"I regret my purchase, can I return it?"	`check_return_window` + `consult_policies("trocas_devolucoes")`	CSV (dates) + manual §4
"How do warranty/shipping/exchanges work?"	`consult_policies`	manual
"Where are you? Are you open?"	`get_store_info`	manual §1–2 + clock
Complaint, defect, unresolved case	`escalate_to_human`	—
Strings, cake recipes, poems...	none — decline and redirect	prompt

Assumptions

The brief says to assume reasonable interpretations and document them:

1. **Contradictions in the manual.**
2. *Phone/WhatsApp:* §1.2 says (67) 3341-4444; §7 says (67) 3321-4500. I used the company data table (§1.2).
3. *E-mail:* the §1.2 table shows `contato@empóriodamusica…` (with an accent, invalid); I used `contato@emporiodamusica.com.br` (manual cover).
4. *Minimum installment:* the §3 table says "R$ 100" for any credit card; §3.1 details bands (up to 3x: no minimum except installments under R$ 50; 4–6x: R$ 80; 7–12x: R$ 100). I adopted **§3.1** (the detailed rule) and read "except below R$ 50" as a R$ 50 minimum installment in the 2–3x band.
5. **Meaning of order statuses** is not defined in the manual. I assumed: `pending` = received/awaiting confirmation; `confirmed` = confirmed/being prepared (no tracking); `shipped` = dispatched; `delivered`; `cancelled`.
6. **Delivery date** does not exist in the CSVs. I use `estimated_delivery` as a proxy to count the 7 days of regret period and the 90 days of legal warranty (and the agent says it is an estimate). The 30-day defect exchange counts from the **purchase date**, as §4.2 states.
7. **Purchase channel:** the right of regret (§4.1) applies to *online* purchases; I assumed the CSV orders are online (they have tracking/carrier).
8. **"Campo Grande metropolitan region"** is not defined (and the data has no ZIP codes), so I **do not compute shipping**: the agent quotes the manual's rules (free above R$ 500, R$ 35 fee,

PAC/SEDEX/Jadlog lead times) and routes special quotes (e.g. drum kits) to the team.
9. **Promotions:** only `is_active = 1` counts; if several are active on one product, the highest discount wins (not cumulative). No active promotion currently falls on an unavailable product.
10. **Clock:** the CSVs are a snapshot from ~March 2026. In real use the agent uses the current date (`America/Campo_Grande` time zone); for demos consistent with the data, `AGENT_NOW` pins the date. Consequence: with the real date, "shipped" orders with an old estimate show as late (desired behavior: apologize and escalate).
11. **Holidays** are not considered for "open now?" (the agent says so).
12. **Partial catalog:** the store claims 300+ items and to sell wind and orchestral string instruments, but the CSVs hold 65 products and the wind/orchestral categories are empty. When a search comes back empty, the agent says "I couldn't find it in the system" and offers the team — it **does not claim the store doesn't sell it**.

Data issues found

Handled or flagged in code (and covered by tests where possible):

Issue	Handling
Swapped descriptions on basses, drum kits and keyboards (e.g. "Music Man Bass 1X" described as a Fender Jazz Bass; "Yamaha Drum Kit 1" as Pearl Export; "Korg keyboard" as a Roland with 88 keys while specs say 61; a bass described as "5-string" with spec "4") — 7 products	detected by a brand/model heuristic (`description_is_reliable`); the description is **omitted** and the agent describes the item with name, price and specs only
Duplicate e-mail across 2 customers (`bruno.martins@jmail.com`, IDs 11 and 43)	requires the order number to disambiguate; never exposes one account's data by mistake
`total_brl` ≠ sum of items on orders 3 (R$ 3,498 vs 3,450) and 20 (R$ 1,488 vs 1,400)	the order's `total_brl` is used (it may include discount/shipping); left as an open item to confirm with the store
A `coming_soon` product (id 130) appears in an already **delivered** order (order 3) and has stock 2	treated as "coming soon — not available for purchase" in the catalog
A `discontinued` product with stock > 0 (id 136, stock 13)	`status` prevails: unavailable
Out-of-order IDs / name typos ("Cedr")	kept as is (the name is the customer-facing identifier)
Wind and orchestral string categories with no products	see assumption 9
Nearly all promotions `is_active = 0` (21 of 25) and one product with 2 old promotions (id 96, 18% and 40%)	only active ones count; ended ones only serve to explain "that one is over"

Tests

`python -m pytest -q` → **49 tests**, no network or key. They cover: catalog search and filters, active vs. ended promotions, PIX vs. promotional price, installments, identity verification (including the duplicate e-mail and the generic failure message), return deadlines at different dates, opening

hours, PDF parsing and retrieval, and the agent loop with a **fake LLM** (tool round-trip, step cap, provider error, history window, input truncation).

What the tests **do not** cover is the *real model's answer quality* (tool choice, tone). For that, see "What I would do with more time".

Example interactions

`examples/` holds 5 conversations covering: (1) catalog with filters, (2) price + active/ended promotion + sold out, (3) order status with verification and delay, (4) return by regret applying policy and deadlines, (5) address/hours + accessory + out-of-scope request. Scenarios 2–4 are "non-trivial" (real-time data + manual rules). The conversations are in Portuguese, as the agent is.

> **Note:** the versioned transcripts are **illustrative** (labeled at the top of each file): tool calls and results are real, but the assistant's lines were written by hand. Run `python scripts/record_examples.py` with your `.env` to generate them with the configured model.

Known limitations

- **No automated evaluation of the LLM** (only deterministic tests); the prompt was designed, not empirically optimized.
- **Weak authentication by prototype design:** e-mail/phone as a "secret" is not strong; in production I would use the sender's WhatsApp number (already authenticated by the channel) + OTP, and rate limiting against enumeration.
- **Escalation tickets** go to a local file (`var/escalations.jsonl`), with no CRM/helpdesk integration; the "24 business hours" follow-up is only communicated.
- **No shipping calculation** (no ZIP code) and **no order creation/change/cancellation**: the agent is read-only.
- **Catalog and stock are read from CSV** at process start (no true "real time"; in production it would be a database/API).
- **Language:** optimized for pt-BR. **One customer per process** (CLI): no concurrency.
- **5-letter-prefix stemming** is crude (could conflate distinct terms in a larger catalog).
- **Customer identity:** the agent does not know who it is talking to until they identify themselves.

What I would do with more time

1. **An evaluation suite (evals)** with ~50 annotated conversations (tool accuracy, no price/promotion hallucination, correct refusals) running in CI against 2–3 models, to choose model/prompt with data.
2. **WhatsApp interface** (Cloud API/Twilio webhook) using the sender's phone as identity; history in SQLite/Redis; a real human **handoff** (CRM/helpdesk) with context.
3. **Streaming + output guardrails:** check that every R$ value in the answer appears in the tool results (a numeric-hallucination detector).

4. **Better recommendations:** qualifying questions (level, style, budget) and side-by-side product comparison.
5. **Data:** fix the records at the source (swapped descriptions, divergent totals), load the full catalog, add real delivery date and carrier to orders.
6. **Embeddings** for the policy fallback if the manual grows; manual versioning.
7. **Observability:** tracing (e.g. Langfuse/OpenTelemetry) of tools, latency and cost per conversation.
8. **ZIP-code shipping** (Correios/Jadlog API) and **multi-item cart** calculations.

Use of code assistants

Tool used: **Claude (Anthropic)**, in agentic mode (reading the files, running code and tests in the environment).

- **Understanding the data:** the assistant inspected the CSVs and the PDF; the findings in the data-issues section (swapped descriptions, duplicate e-mail, divergent totals) came from that and were checked by running queries over the files.
- **Design:** alternatives were compared (SQL agent, function calling, vector RAG) and function calling with business tools was chosen.
- **Implementation and tests:** the assistant wrote the modules and the pytest tests; failures surfaced by the tests themselves (PDF parsing, "kala" vs. "kalani" search, BM25 synonyms) were fixed afterwards.
- **Documentation:** this README was drafted with the assistant.
- **Important caveat:** the development environment had no LLM key, so the assistant's lines in `examples/` were written by hand (flagged at the top of each file) and the flow with a real model was not exercised. Run `python scripts/record_examples.py` to generate the conversations with the configured model.